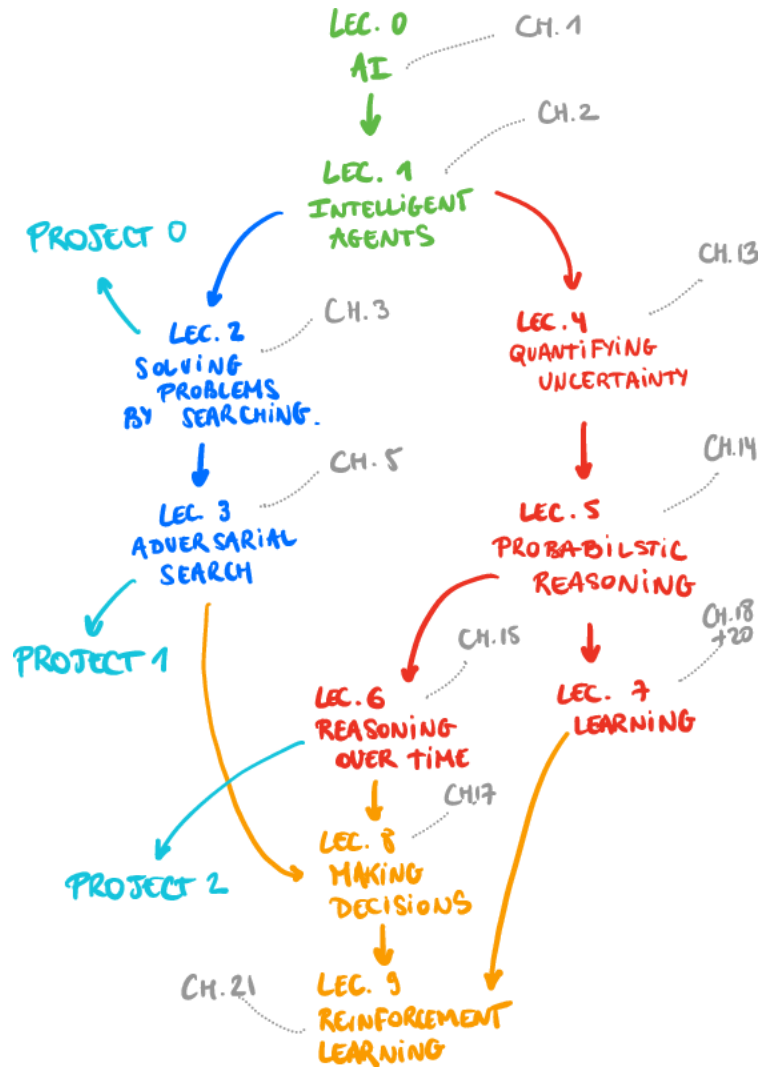


# Introduction to Artificial Intelligence

Lecture 2: Solving problems by searching

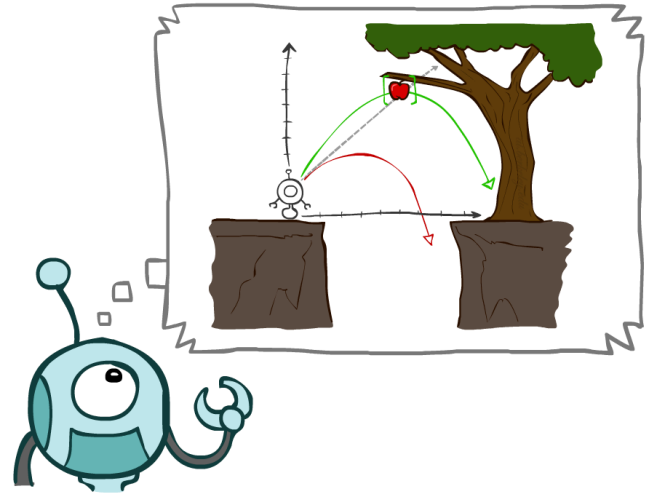
Prof. Gilles Louppe

[g.louppe@uliege.be](mailto:g.louppe@uliege.be)



# Today

- Planning agents
- Search problems
- Uninformed search methods
  - Depth-first search
  - Breadth-first search
  - Uniform-cost search
- Informed search methods
  - A\*
  - Heuristics



# Planning agents

# Reflex agents

A **reflex agent** acts on the current percept alone,  $a_t = \pi(e_t)$ . It may keep a model of the world, but it ignores the percept history and the future consequences of its actions.

Reflexes are enough to act rationally when the environment is **fully observable**, **deterministic** and **known**.

For complex tasks, they are difficult to implement: the action with the best long-term outcome is more naturally expressed as planning.

# Problem-solving agents

A **problem-solving agent** is a **goal-based agent** (Lecture 1): given goal states  $G \subseteq \mathcal{S}$ , it **searches** its transition model for an action sequence that reaches one of them, instead of reacting to the current percept.

Assumptions on the environment:

- **single agent**: no other agent acts on the environment;
- **fully observable**: the percept reveals the state,  $e_t = s_t$ ;
- **deterministic**:  $s_{t+1} = \text{result}(s_t, a_t)$ ;
- **known**: the agent knows **result**.

The agent can therefore plan the whole sequence before acting.

**function** SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

**persistent:** *seq*, an action sequence, initially empty

*state*, some description of the current world state

*goal*, a goal, initially null

*problem*, a problem formulation

*state*  $\leftarrow$  UPDATE-STATE(*state*, *percept*)

**if** *seq* is empty **then**

*goal*  $\leftarrow$  FORMULATE-GOAL(*state*)

*problem*  $\leftarrow$  FORMULATE-PROBLEM(*state*, *goal*)

*seq*  $\leftarrow$  SEARCH(*problem*)

**if** *seq* = *failure* **then return** a null action

*action*  $\leftarrow$  FIRST(*seq*)

*seq*  $\leftarrow$  REST(*seq*)

**return** *action*

## Offline vs. Online solving

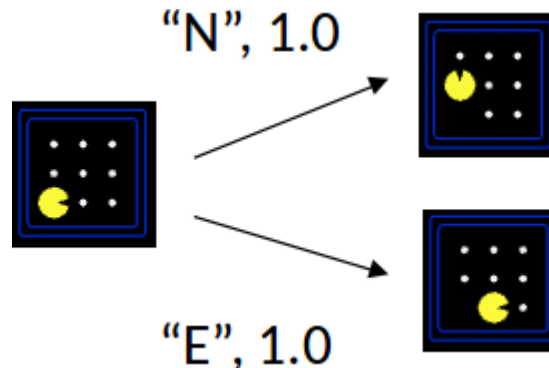
- **Offline**: the agent computes the whole sequence  $a_{1:T}$  from  $s_1$ , then executes it "eyes closed", ignoring its percepts  $e_{1:t}$ .
- **Online**: the agent recomputes what to do at each step from what it has perceived,  $a_t = \pi(e_{1:t})$ . That is a policy, not a plan.

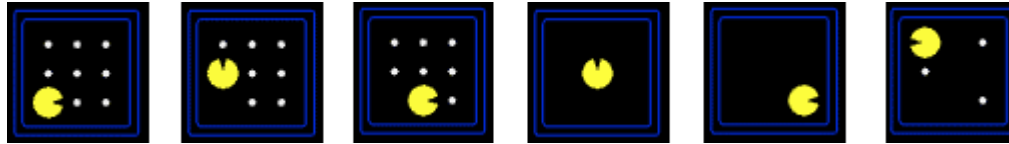
# Search problems

# Search problems

A **search problem** is defined by

- the **states**  $s \in \mathcal{S}$  of the agent and its environment, and the **initial state**  $s_1 \in \mathcal{S}$ ;
- the **actions**  $\text{actions}(s) \subseteq \mathcal{A}$  available in a state  $s$ ;
- a **transition model**  $s' = \text{result}(s, a)$ , the state that results from taking  $a$  in  $s$ .
  - We say that  $s'$  is a **successor** of  $s$  if some applicable action leads from  $s$  to  $s'$ .





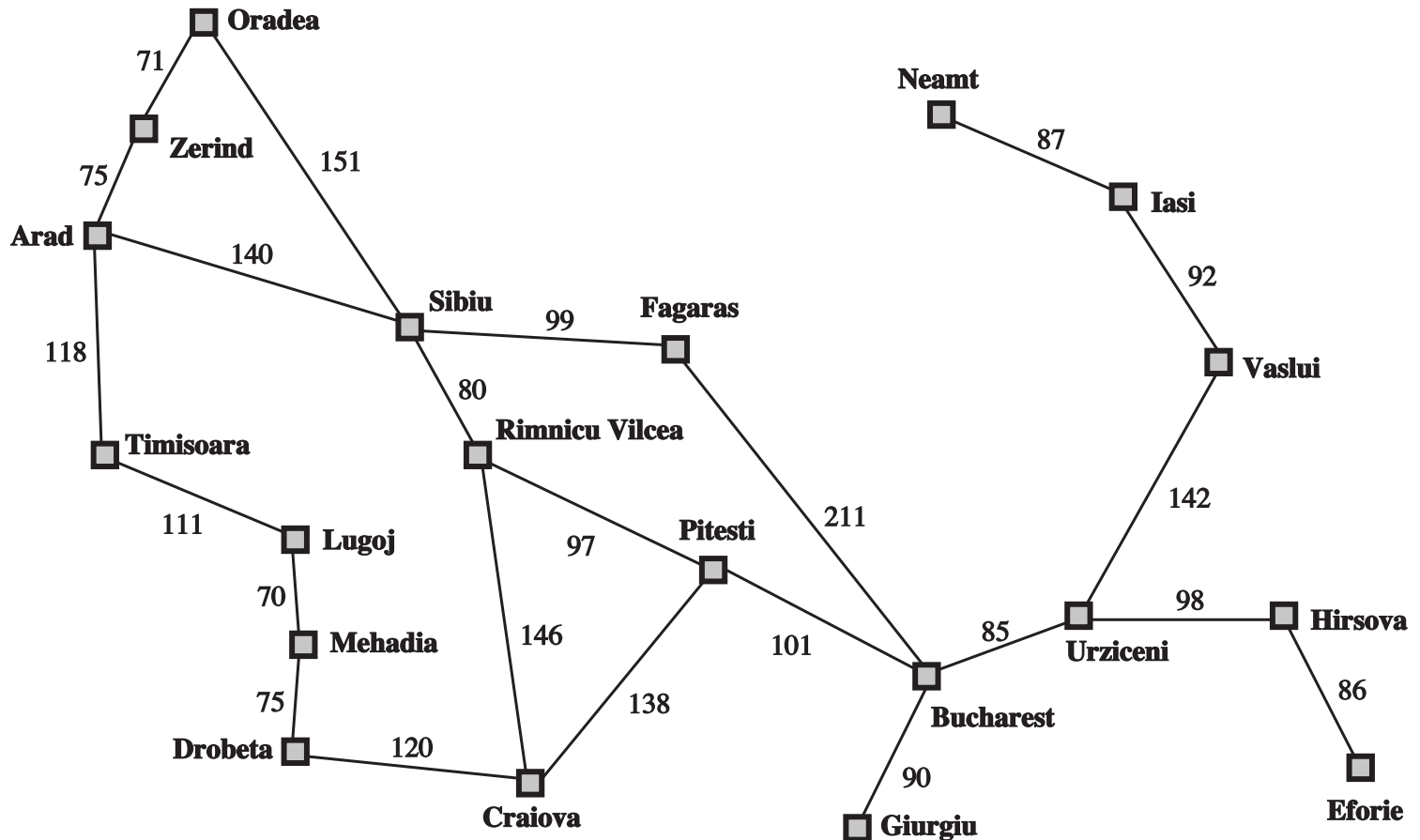
- Together,  $s_1$ , **actions** and **result** define the **state space**, the set of states reachable from  $s_1$  by any sequence of actions.
  - It forms a directed graph: nodes are states, edges are actions.
  - A path is a sequence of states connected by actions.
- The **goal states**  $G \subseteq \mathcal{S}$  are those in which the problem is solved. The **goal test** checks whether  $s \in G$ .
- The **step costs**  $c(s, a, s') > 0$  give the cost of taking  $a$  in  $s$  to reach  $s'$ . The **path cost** is their sum along a path.

A **solution** is an action sequence  $a_{1:T}$  such that  $s_{t+1} = \text{result}(s_t, a_t)$  for  $t = 1, \dots, T$  and  $s_{T+1} \in G$ . Its path cost is

$$\sum_{t=1}^T c(s_t, a_t, s_{t+1}).$$

An **optimal solution** is a solution of minimal path cost.

## Example: Traveling in Romania



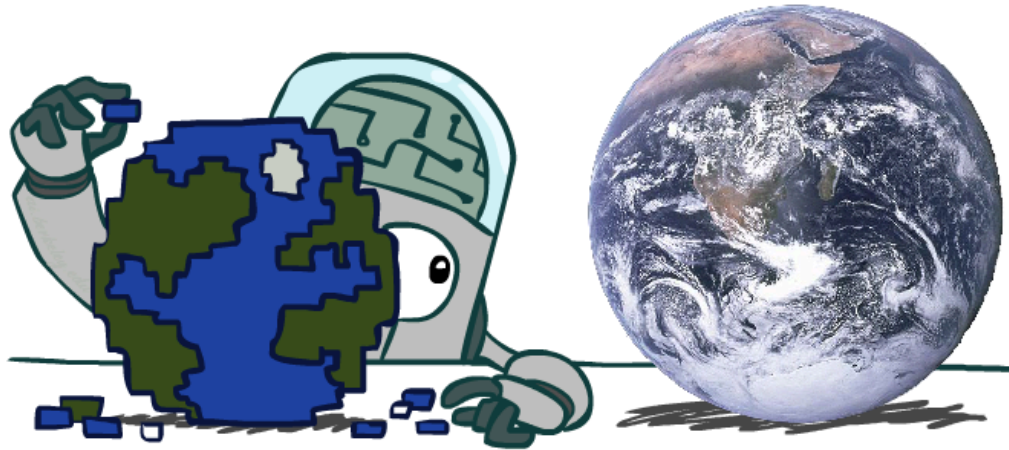
*How to go from Arad to Bucharest?*

- Representation of states: the city we are in.
  - $s \in \{\text{in}(\text{Arad}), \text{in}(\text{Bucharest}), \dots\}$
- Initial state = the city we start in.
  - $s_1 = \text{in}(\text{Arad})$
- Actions = Going from the current city to the cities that are directly connected to it.
  - $\text{actions}(s_1) = \{\text{go}(\text{Sibiu}), \text{go}(\text{Timisoara}), \text{go}(\text{Zerind})\}$
- Transition model = The city we arrive in after driving to it.
  - $\text{result}(\text{in}(\text{Arad}), \text{go}(\text{Zerind})) = \text{in}(\text{Zerind})$
- Goal states: being in Bucharest.
  - $G = \{\text{in}(\text{Bucharest})\}$
- Step cost: distances between cities.

# Selecting a state space

The real world is absurdly **complex**.

- The **world state** includes every last detail of the environment.
- A **search state**  $s \in \mathcal{S}$  keeps only the details needed for planning.

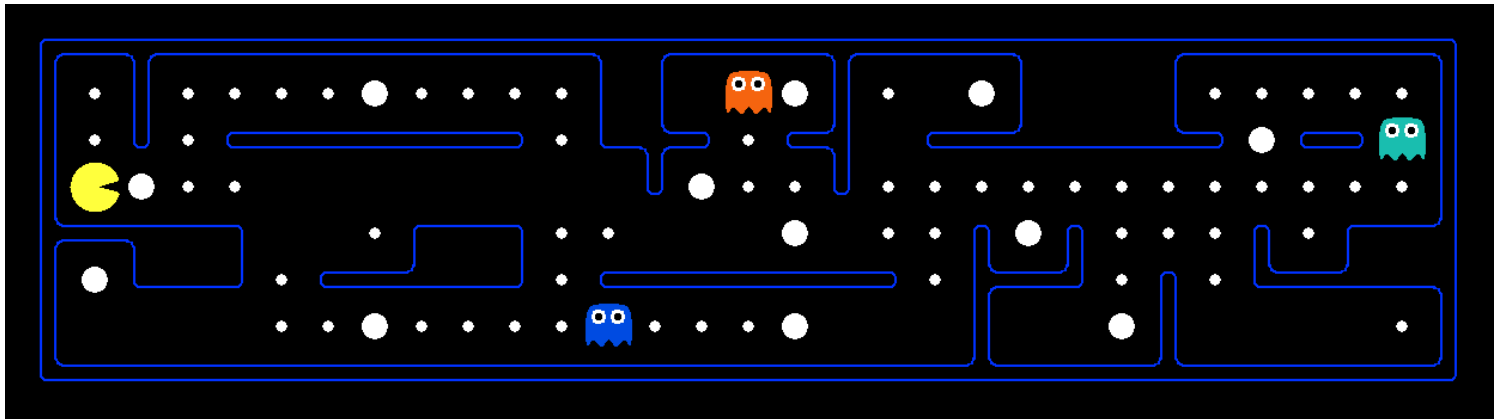


Search problems are **models**.

## Example: eat-all-dots

In a maze with  $k$  dots,

- states: a position and the dots left,  $s = ((x, y), d)$  with  $d \in \{0, 1\}^k$ ;
- actions:  $\mathcal{A} = \{N, S, E, W\}$ ;
- transition model:  $\text{result}(s, a)$  moves Pacman one cell, and clears the dot it finds there;
- goal states:  $G = \{((x, y), d) : d = \mathbf{0}\}$ ;
- step cost: 1 per move.



## State space size

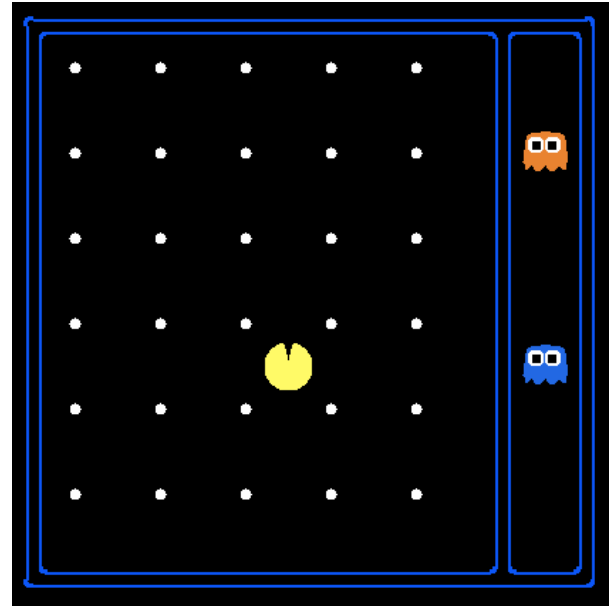
The maze on the right has 120 cells, 30 dots and 2 ghosts. A **world state** holds

- the position of Pacman, 120 values;
- the dots left,  $2^{30}$ ;
- the positions of the ghosts,  $12^2$ ;
- the direction Pacman faces, 4;

hence  $120 \times 2^{30} \times 12^2 \times 4 \approx 7 \times 10^{13}$

world states, against

$|\mathcal{S}| = 120 \times 2^{30} \approx 1.3 \times 10^{11}$  for eat-all-dots.

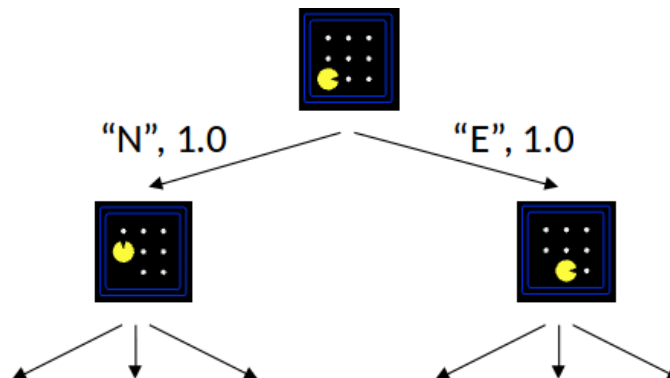


# Tree search algorithms

We look for a path from  $s_1$  to  $G$  in the state space.

To find one, search **grows candidate paths** from  $s_1$ , one action at a time, until one of them reaches  $G$ .

The candidates form the **search tree**: its root is  $s_1$ , its branches are actions, and a **node  $n$**  is a path, of last state  $s(n)$  and cost  $g(n)$ . A state reached by several paths sits in several nodes.



**function** TREE-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

*fringe*  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

**loop do**

**if** *fringe* is empty **then return** failure

*node*  $\leftarrow$  REMOVE-FRONT(*fringe*)

**if** GOAL-TEST(*problem*, STATE[*node*]) **then return** *node*

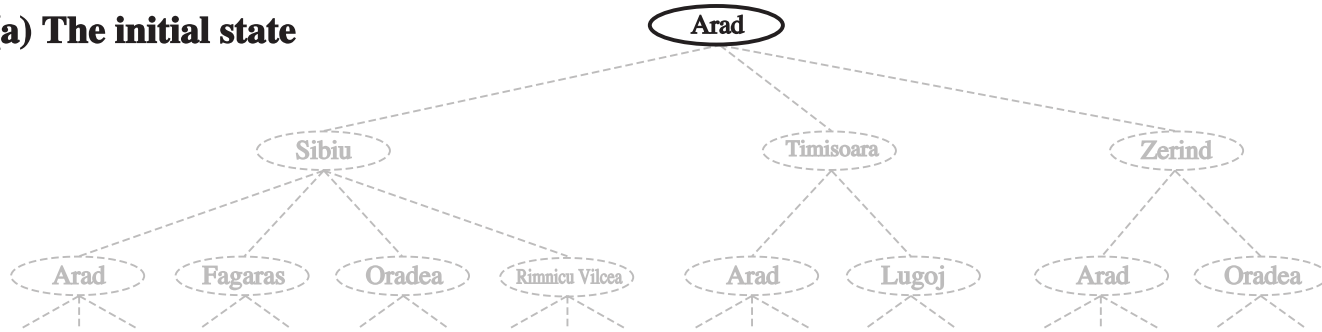
*fringe*  $\leftarrow$  INSERTALL(EXPAND(*node*, *problem*), *fringe*)

## Important ideas

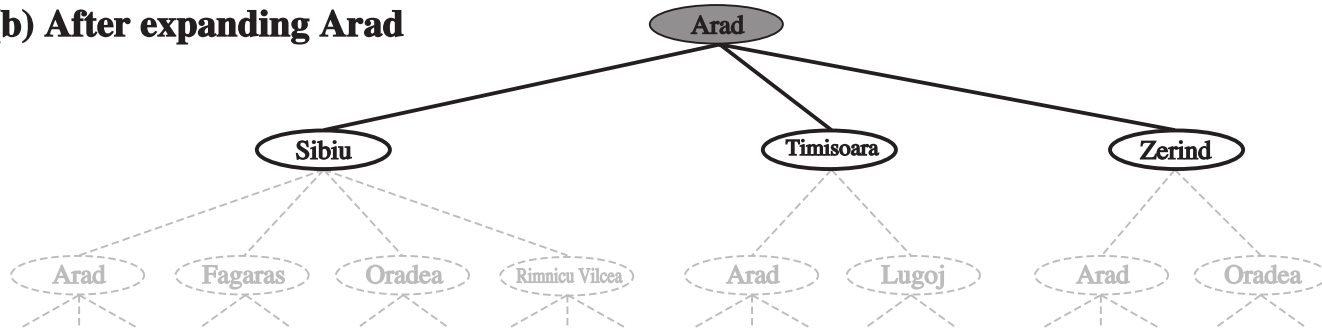
- **Fringe** (or frontier): the leaves of the tree, generated but not expanded.
- **Expansion**: replacing a leaf by its children.
- **Strategy**: which leaf to expand next.

The tree is far too large to build. Which leaves do we expand to reach  $G$ , and how do we expand as few as possible?

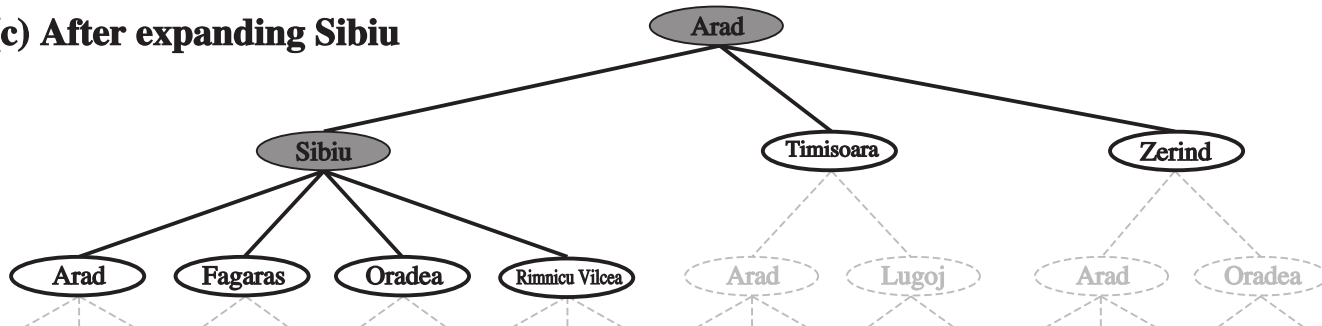
**(a) The initial state**



**(b) After expanding Arad**



**(c) After expanding Sibiu**



# Uninformed search strategies

# Uninformed search strategies

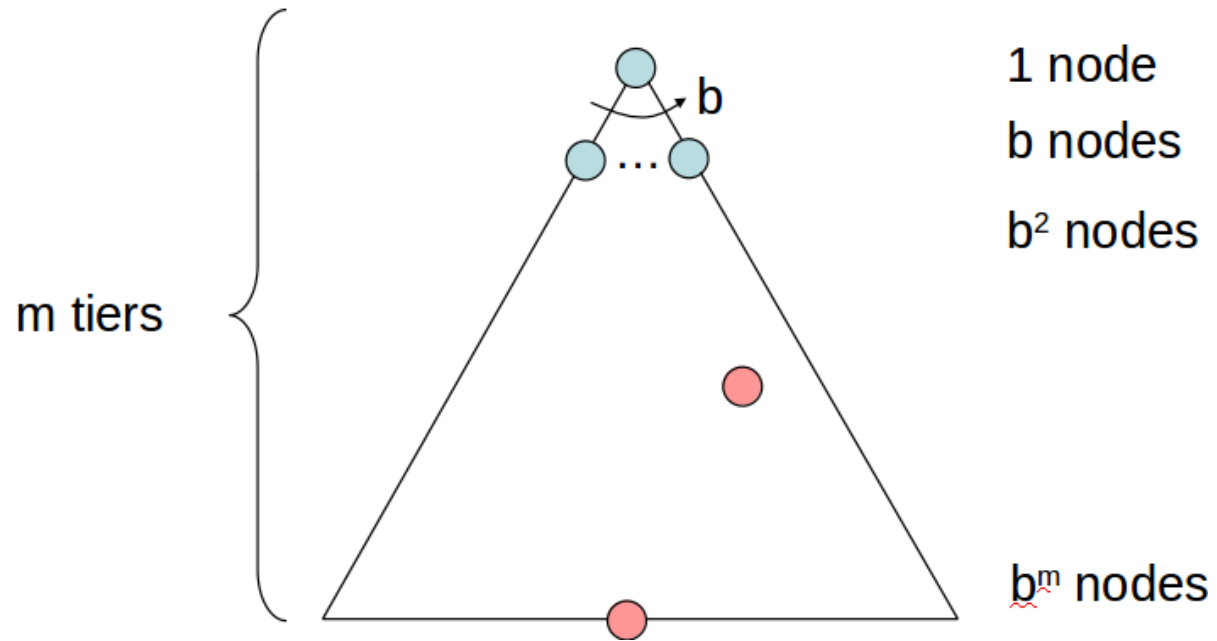
**Uninformed** search strategies use only the information available in the problem definition. They do not know whether a state looks more promising than some other.

## Strategies

- Depth-first search
- Breadth-first search
- Uniform-cost search
- Iterative deepening

# Properties of search strategies

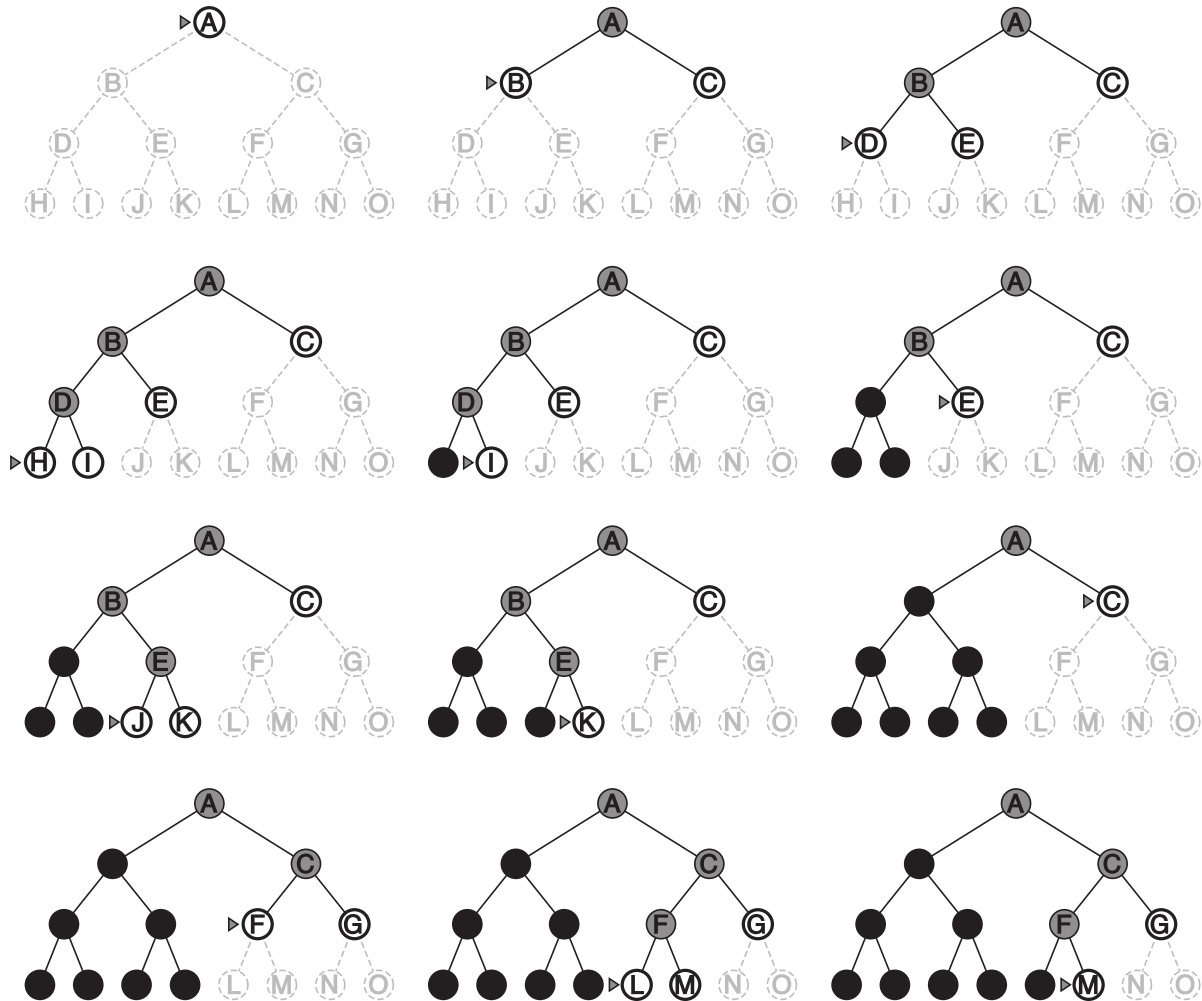
- A strategy is defined by picking the **order of expansion**.
- Strategies are evaluated along the following dimensions:
  - **Completeness**: does it always find a solution if one exists?
  - **Optimality**: does it always find the least-cost solution?
  - **Time complexity**: how long does it take to find a solution?
  - **Space complexity**: how much memory is needed to perform the search?
- Time and space complexity are measured in terms of
  - $b$ : maximum branching factor of the search tree
  - $d$ : depth of the least-cost solution
    - the depth of  $s$  is defined as the number of actions from the initial state to  $s$ .
  - $m$ : maximum length of any path in the state space (may be  $\infty$ )

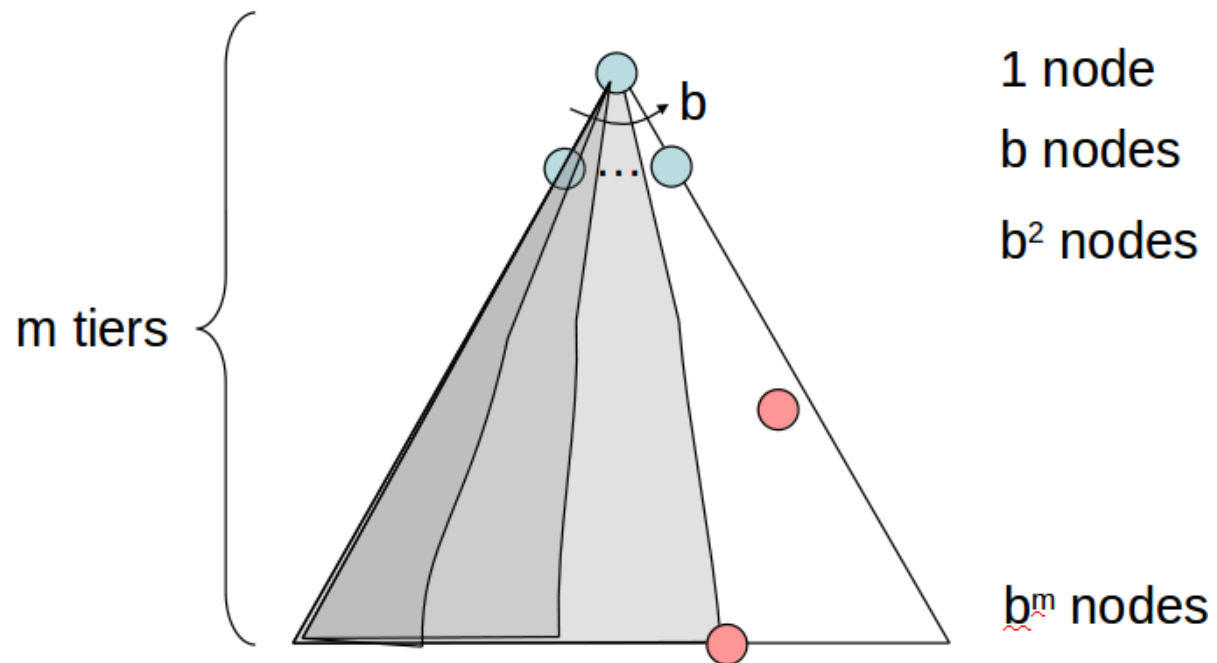


# Depth-first search



- **Strategy:** expand the deepest node in the fringe.
- **Implementation:** fringe is a **LIFO stack**.





## Properties of DFS

- **Completeness:**

- $m$  could be infinite, so only if we prevent cycles (more on this later).

- **Optimality:**

- No, DFS finds the leftmost solution, regardless of depth or cost.

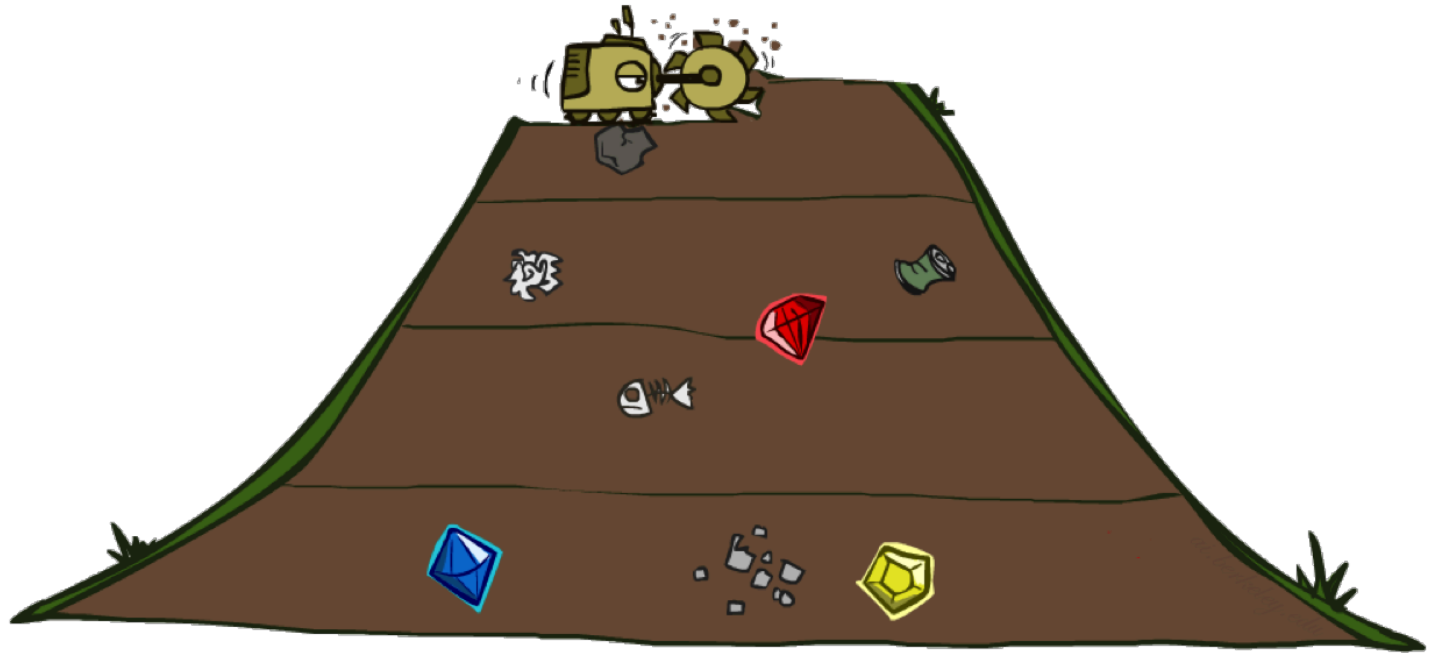
- **Time complexity:**

- May generate the whole tree (or a good part of it, regardless of  $d$ ). Therefore  $O(b^m)$ , which might be much greater than the size of the state space!

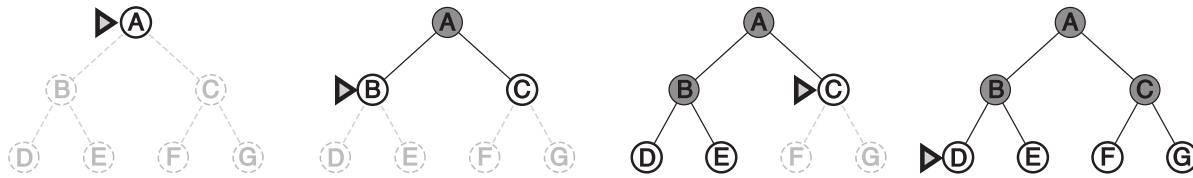
- **Space complexity:**

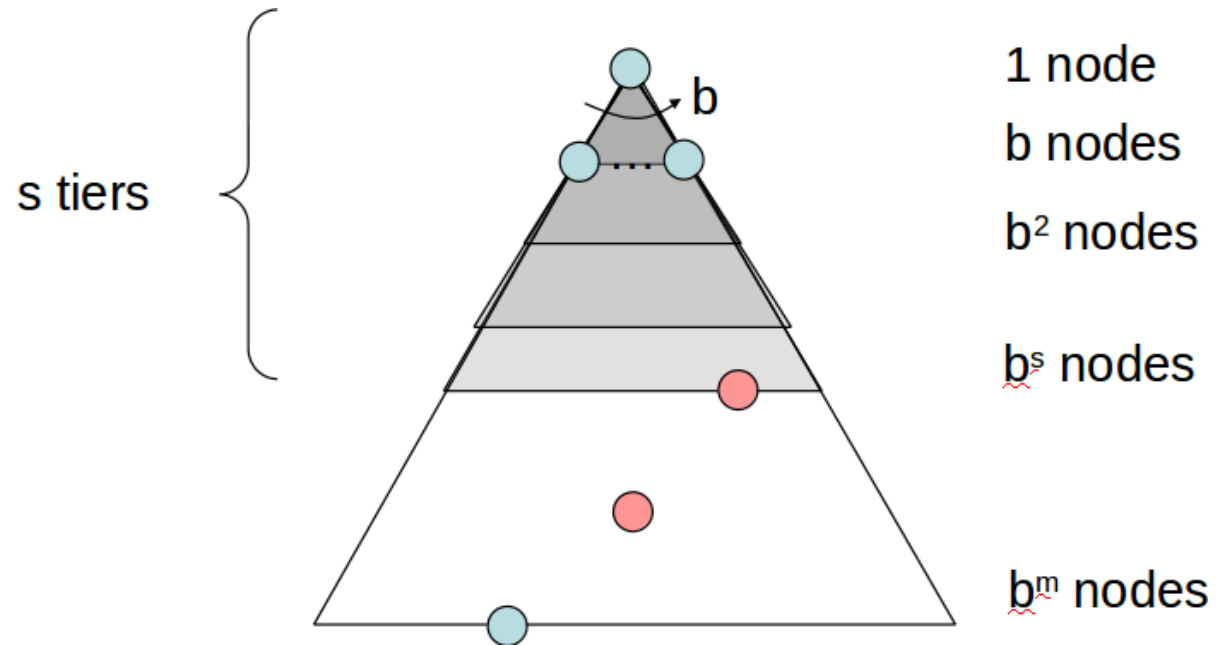
- Only store siblings on path to root, therefore  $O(bm)$ .
- When all the descendants of a node have been visited, the node can be removed from memory.

# Breadth-first search



- **Strategy:** expand the shallowest node in the fringe.
- **Implementation:** fringe is a **FIFO queue**.





# Properties of BFS

- **Completeness:**

- If the shallowest goal node is at some finite depth  $d$ , BFS will eventually find it after generating all shallower nodes (provided  $b$  is finite).

- **Optimality:**

- The shallowest goal is not necessarily the optimal one.
- BFS is optimal only if the path cost is a non-decreasing function of the depth of the node.

- **Time complexity:**

- If the solution is at depth  $d$ , then the total number of nodes generated before finding this node is  $b + b^2 + b^3 + \dots + b^d = O(b^d)$

- **Space complexity:**

- The number of nodes to maintain in memory is the size of the fringe, which will be the largest at the last tier. That is  $O(b^d)$

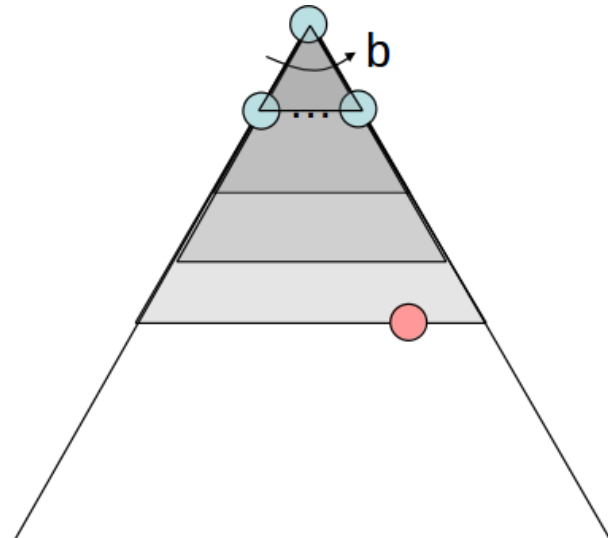
(demo)

# Iterative deepening

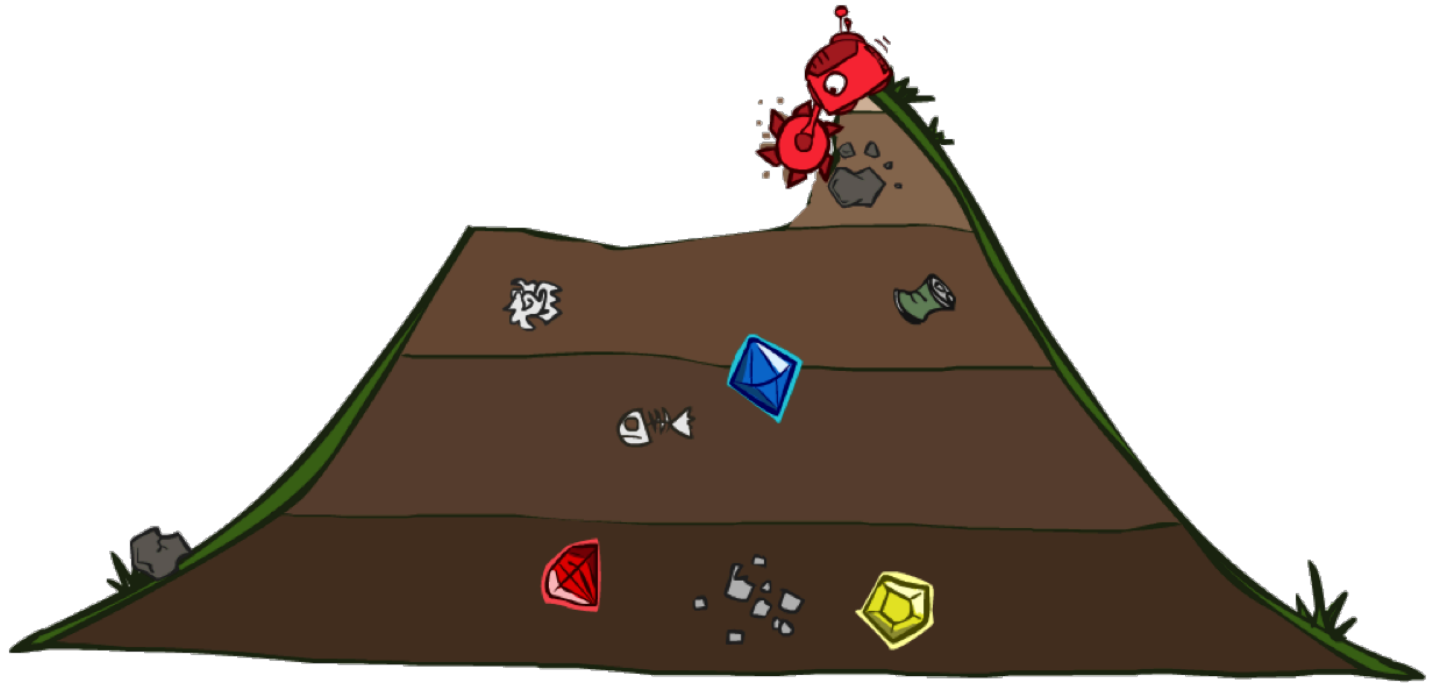
Idea: get DFS's memory with BFS's guarantees, by running DFS with depth limit 1, then 2, then 3, and so on.

It is **complete**, **optimal** when the path cost is non-decreasing with the depth, in  $O(b^d)$  time and  $O(bd)$  memory.

Isn't this process wastefully redundant?

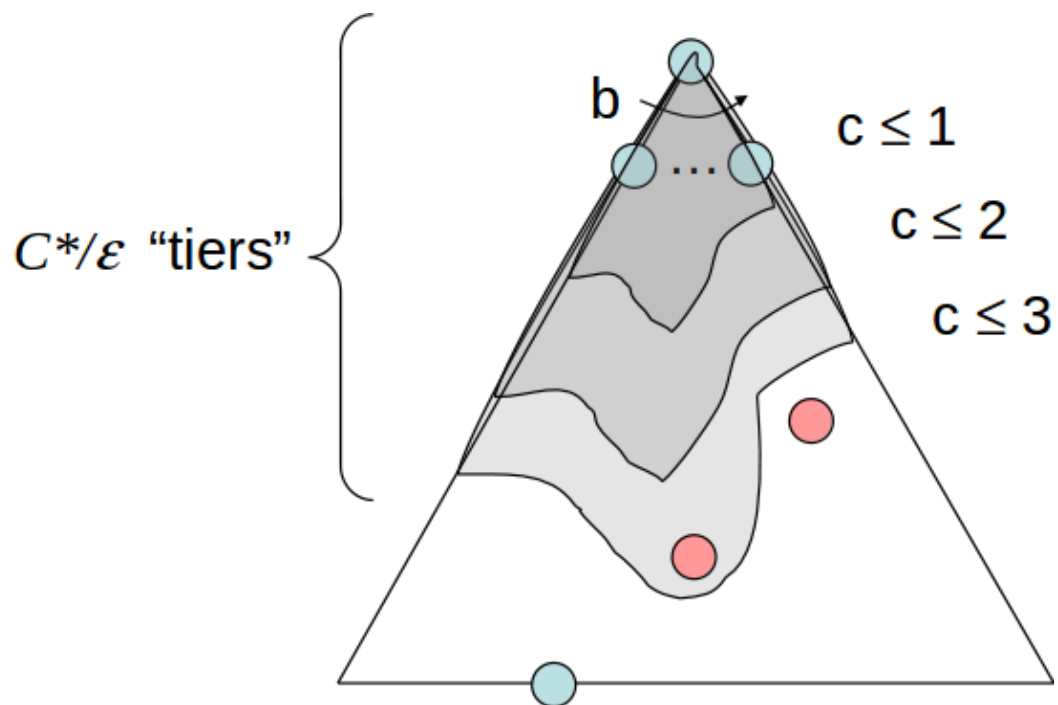


# Uniform-cost search



- **Strategy:** expand the cheapest node in the fringe.
- **Implementation:** fringe is a **priority queue**, using as priority the cumulative cost from  $s_1$  to node  $n$ , that is, over the path  $a_{1:k}$  that  $n$  holds,

$$g(n) = \sum_{t=1}^k c(s_t, a_t, s_{t+1}).$$



## Properties of UCS

- **Completeness:**

- Yes, if the step costs are all such that  $c(s, a, s') \geq \epsilon > 0$ . (Why?)

- **Optimality:**

- Yes, since UCS expands nodes in order of their optimal path cost.

- **Time complexity:**

- Assume  $C^*$  is the cost of the optimal solution and that step costs are all  $\geq \epsilon$ .
- The "effective depth" is then roughly  $C^* / \epsilon$ .
- The worst-case time complexity is  $O(b^{C^* / \epsilon})$ .

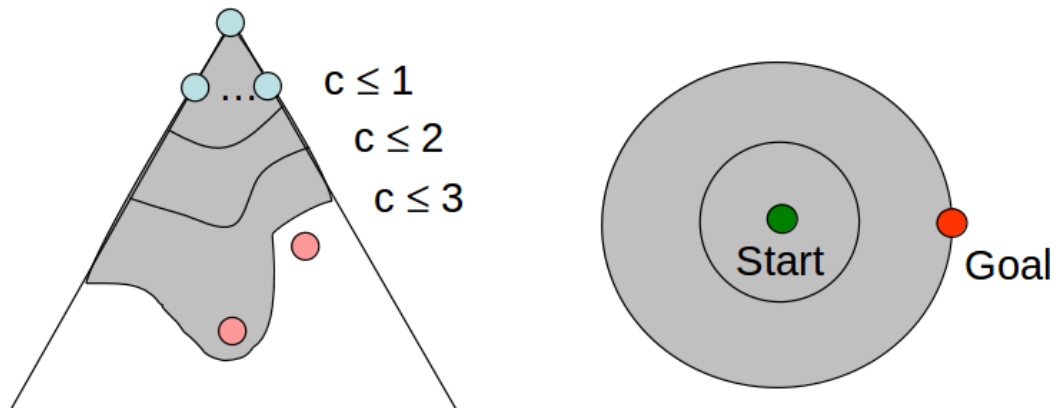
- **Space complexity:**

- The number of nodes to maintain is the size of the fringe, so as many as in the last tier  $O(b^{C^* / \epsilon})$ .

(demo)

# Informed search strategies

# Informed search strategies



One of the **issues of UCS** is that it explores the state space in **every direction**, without exploiting information about the (plausible) location of the goal node.

**Informed** search strategies aim to solve this problem by expanding nodes in the fringe in decreasing order of **desirability**.

- Greedy search
- $A^*$

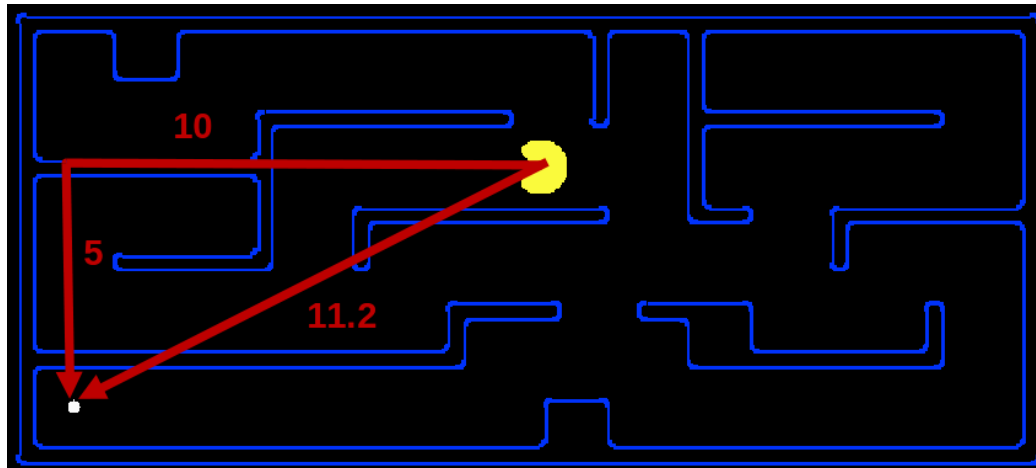
# Greedy search



# Heuristics

A **heuristic** (or evaluation) function  $h(n)$  is:

- a function that **estimates** the cost of the cheapest path from node  $n$  to a goal state;
  - $h(n) \geq 0$  for all nodes  $n$
  - $h(n) = 0$  for a goal state.
- is designed for a **particular** search problem.



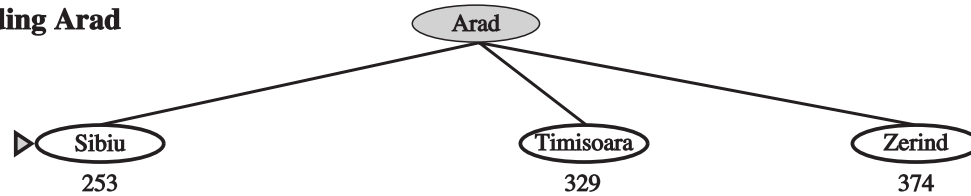
## Greedy search

- **Strategy:** expand the node  $n$  in the fringe for which  $h(n)$  is the lowest.
- **Implementation:** fringe is a **priority queue**, using  $h(n)$  as priority.

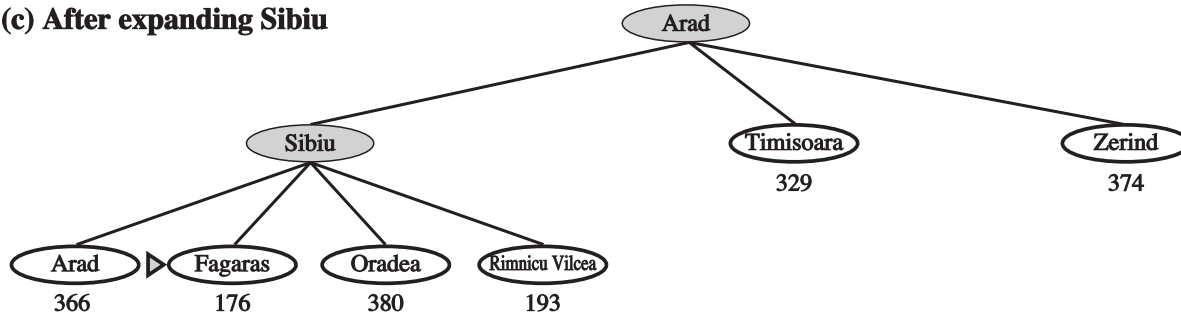
(a) The initial state



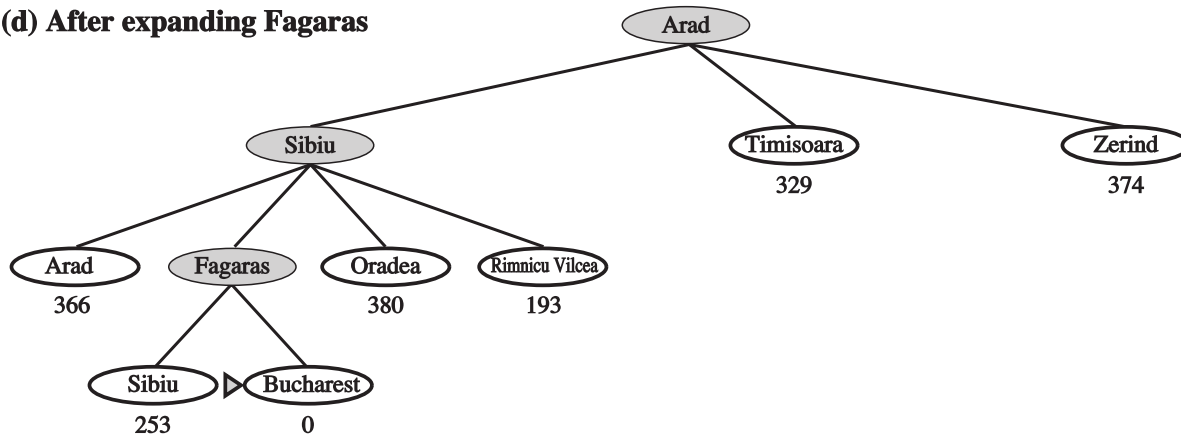
(b) After expanding Arad



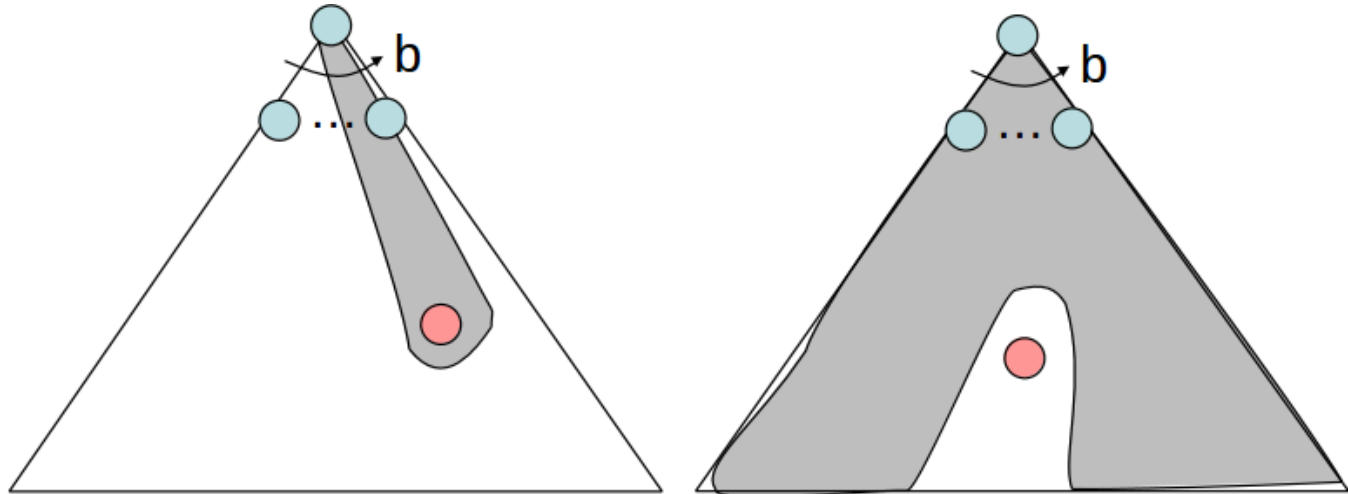
(c) After expanding Sibiu



(d) After expanding Fagaras



$h(n)$  = straight line distance to Bucharest.



At best, greedy search takes you straight to the goal.  
At worst, it is like a badly-guided BFS.

## Properties of greedy search

- **Completeness:**

- No, unless we prevent cycles (more on this later).

- **Optimality:**

- No, e.g. the path via Sibiu and Fagaras is 32km longer than the path through Rimnicu Vilcea and Pitesti.

- **Time complexity:**

- $O(b^m)$ , unless we have a good heuristic function.

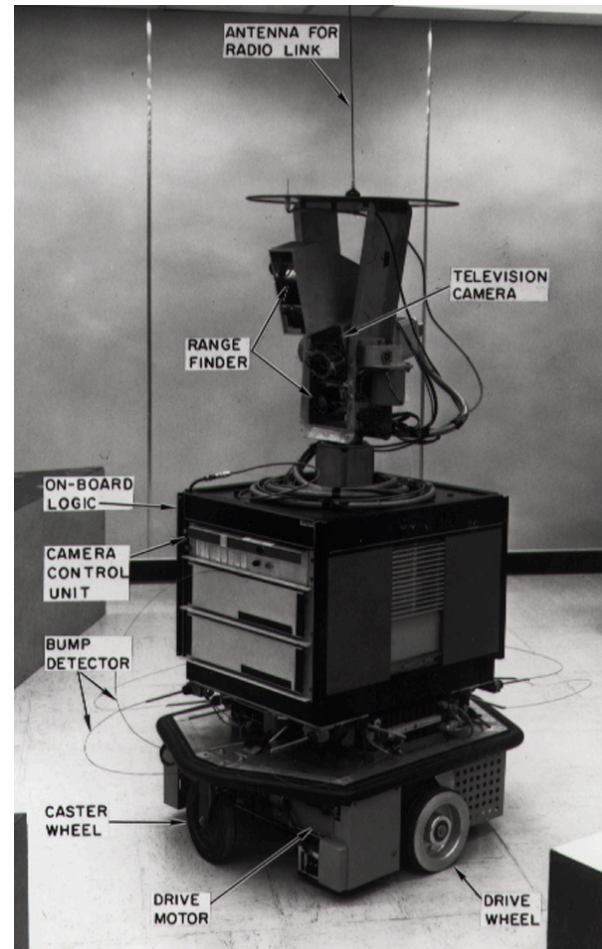
- **Space complexity:**

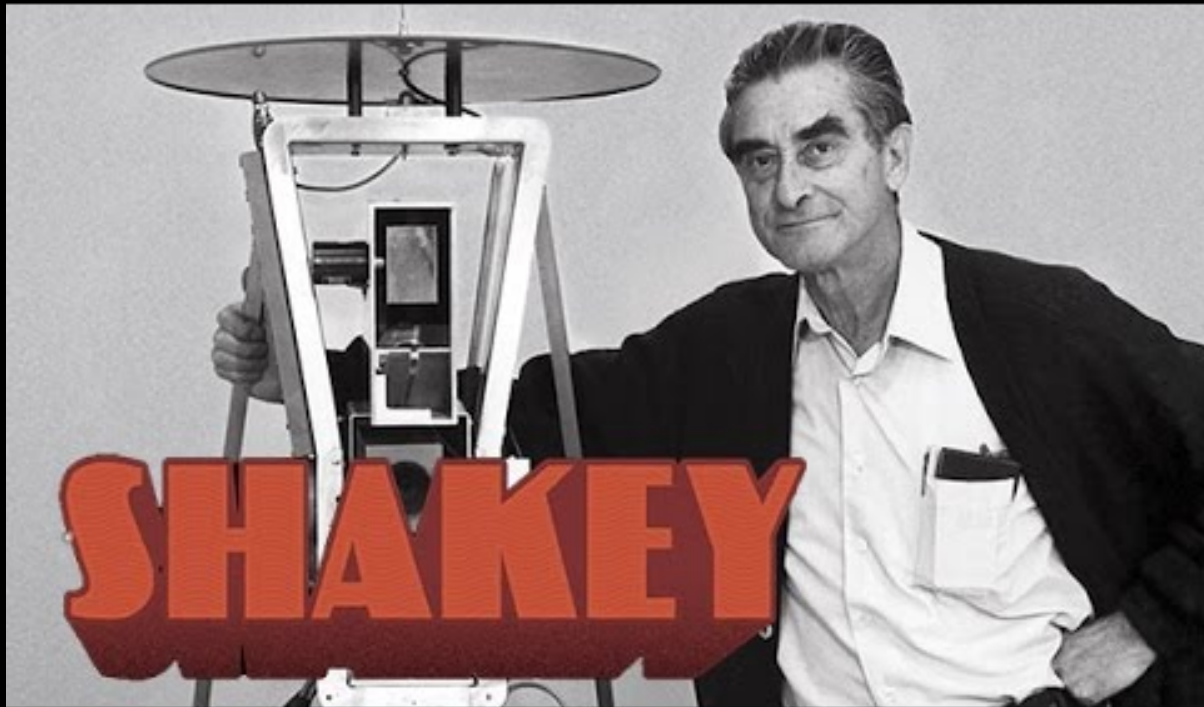
- $O(b^m)$ , unless we have a good heuristic function.



## Shakey the Robot

- A\* was first proposed in **1968** to improve robot planning.
- Goal was to navigate through a room with obstacles.





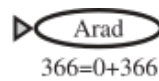
## A\*

- Uniform-cost orders by path cost, or **backward cost**  $g(n)$
- Greedy orders by goal proximity, or **forward cost**  $h(n)$
- **A\*** combines the two algorithms and orders by the sum

$$f(n) = g(n) + h(n)$$

- $f(n)$  is the estimated cost of cheapest solution through  $n$ .

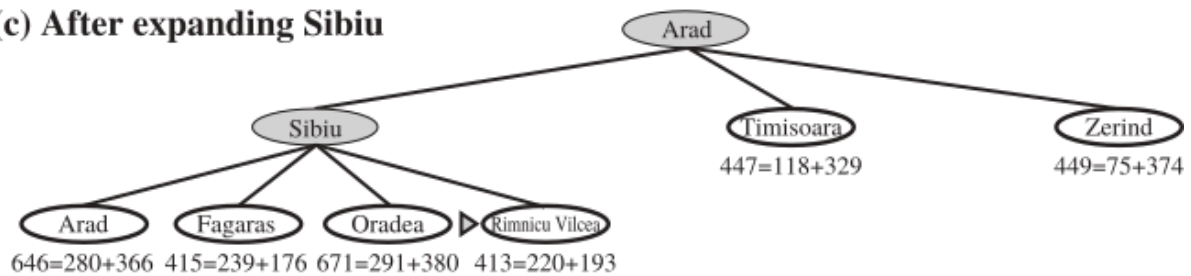
(a) The initial state



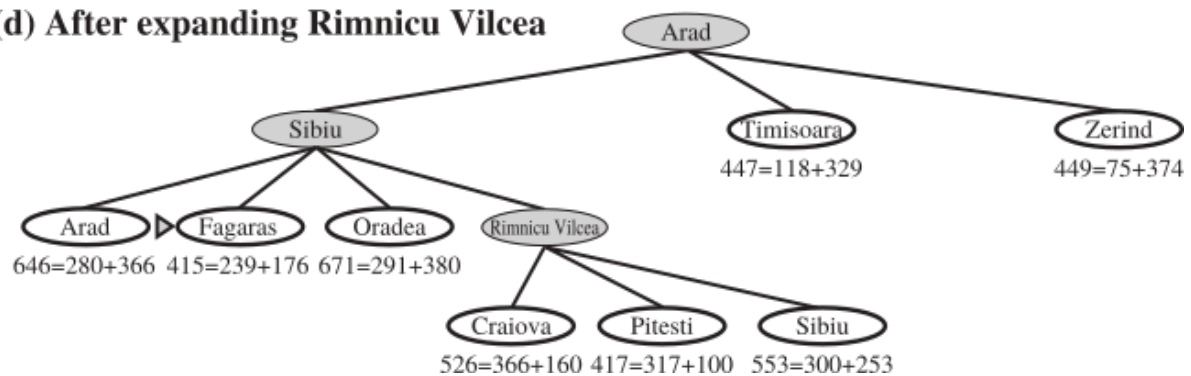
(b) After expanding Arad



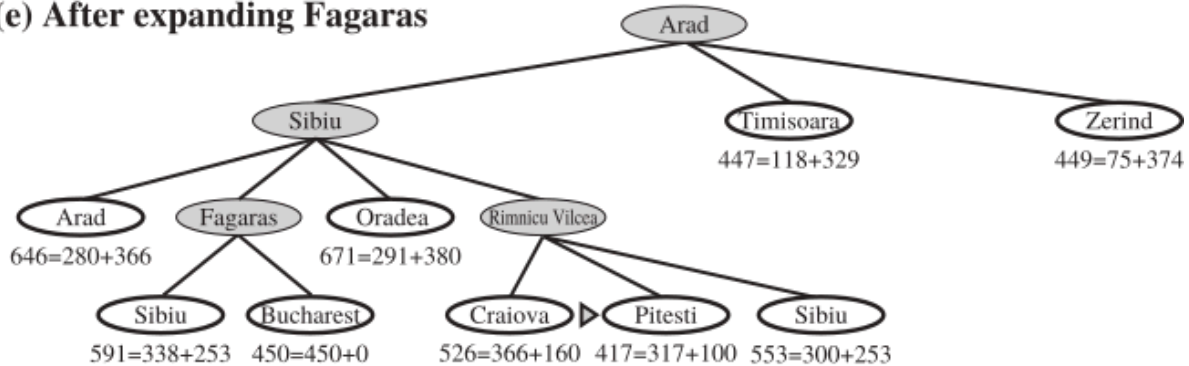
(c) After expanding Sibiu



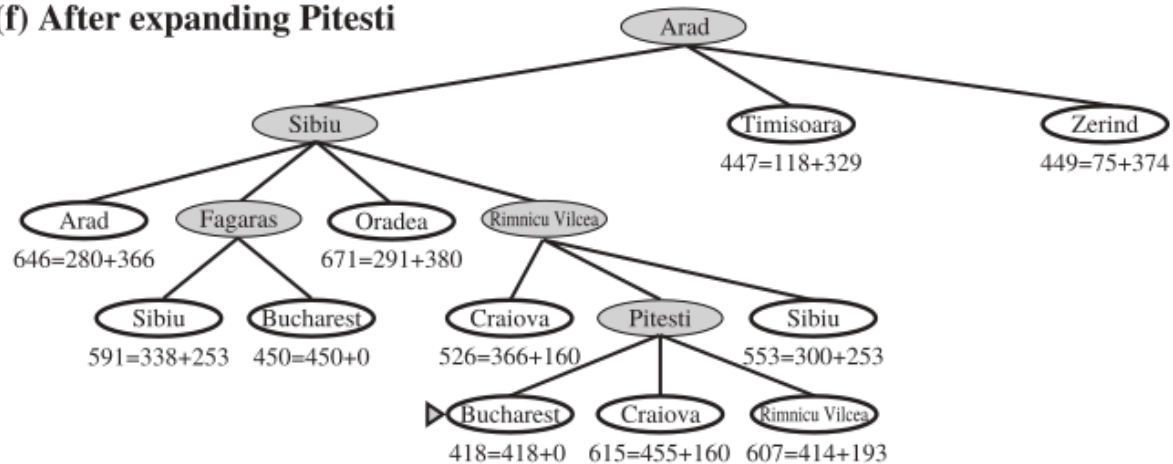
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti



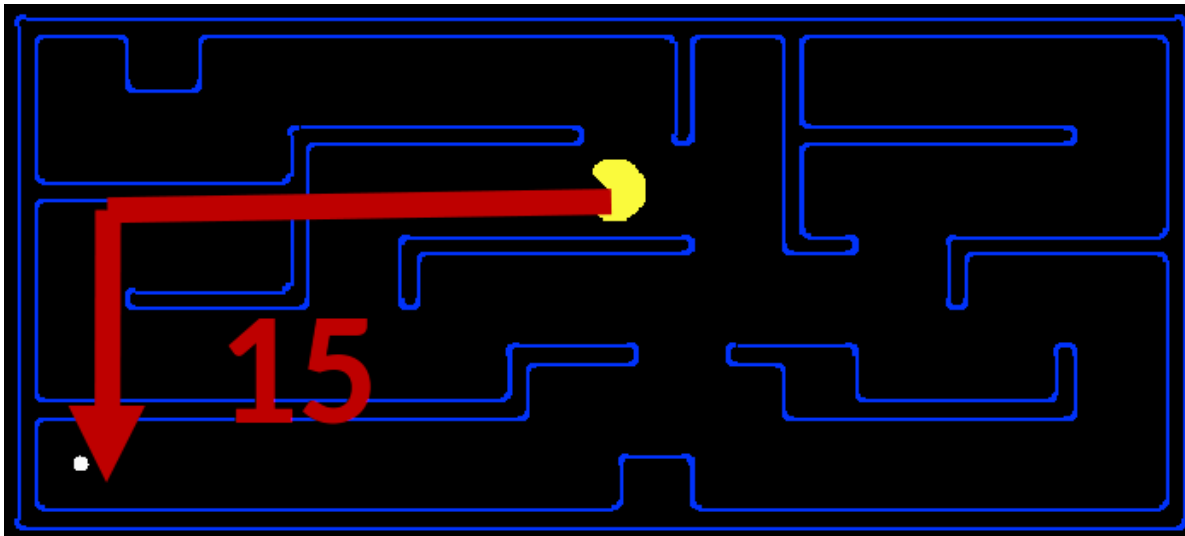
Why doesn't A\* stop at step (e), since Bucharest is in the fringe?

## Admissible heuristics

A heuristic  $h$  is **admissible** if

$$0 \leq h(n) \leq h^*(n)$$

where  $h^*(n)$  is the true cost to a nearest goal.



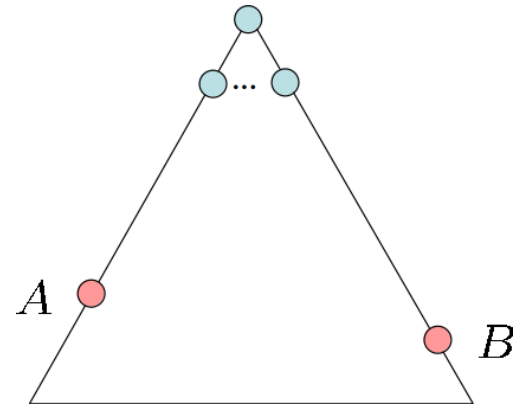
*The Manhattan distance is admissible*

## Optimality of A\*

Assumptions:

- $A$  is an optimal goal node
- $B$  is a suboptimal goal node
- $h$  is admissible

Claim:  $A$  will exit the fringe before  $B$ .

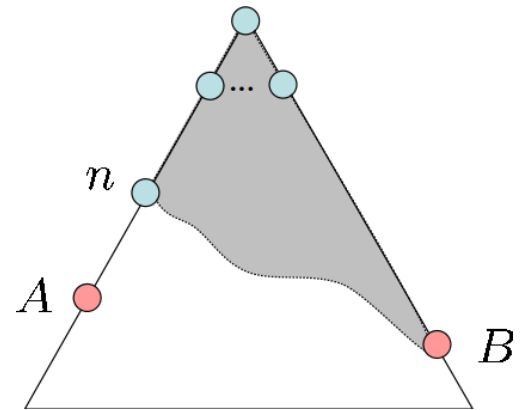


## Proof

Assume  $B$  is on the fringe, and let  $n$  be an ancestor of  $A$  on the fringe. Then

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = g(A) = f(A) < f(B)$$

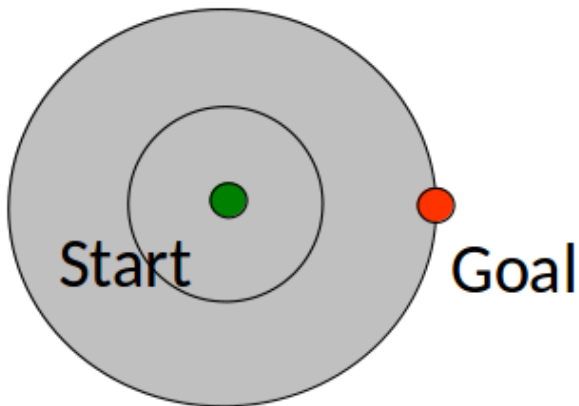
- $h(n) \leq h^*(n)$ , since  $h$  is admissible;
- $g(n) + h^*(n) = g(A)$ , since  $n$  is on an optimal path to  $A$ ;
- $f(A) = g(A)$  and  $f(B) = g(B)$ , since  $h = 0$  at a goal;
- $g(A) < g(B)$ , since  $B$  is suboptimal.



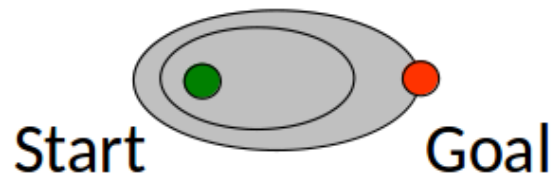
Therefore  $n$  expands before  $B$ . The same holds for every ancestor of  $A$ , hence  $A$  expands before  $B$  and  **$A^*$  is optimal.**

## A\* contours

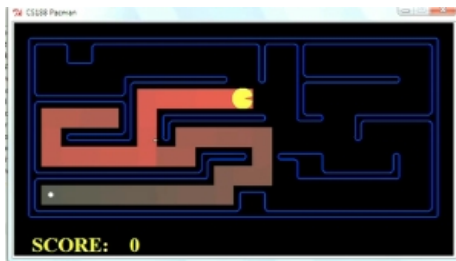
- Assume  $f$ -costs are non-decreasing along any path.
- We can define **contour levels**  $t$  in the state space, that include all nodes  $n$  for which  $f(n) \leq t$ .
- A\* expands every node with  $f(n) < C^*$  and none with  $f(n) > C^*$ : it sweeps the contours outwards until it reaches the goal.



For UCS ( $h(n) = 0$  for all  $n$ ), bands are circular around the start.



For A\* with accurate heuristics, bands stretch towards the goal.



Greedy search



UCS



A\*

(demo)

## Comparison of the strategies

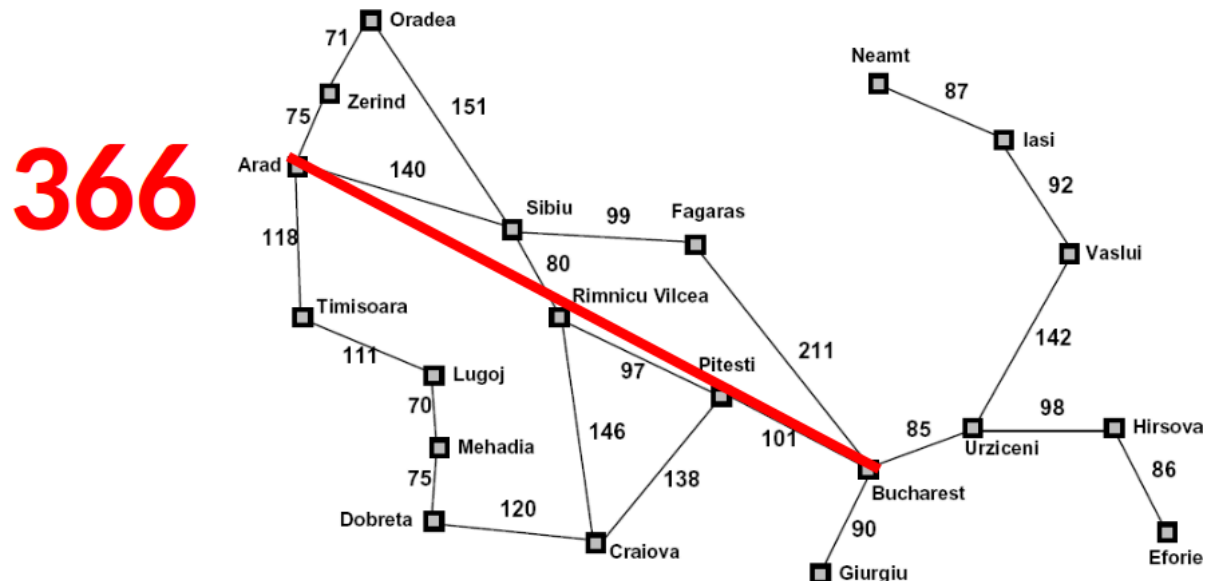
With  $b$  the branching factor,  $d$  the depth of the least-cost solution,  $m$  the maximum path length,  $C^*$  the optimal cost and  $\epsilon$  a lower bound on the step costs:

| Strategy            | Fringe                   | Complete             | Optimal                  | Time                  | Space                 |
|---------------------|--------------------------|----------------------|--------------------------|-----------------------|-----------------------|
| Depth-first         | LIFO stack               | if no cycles         | no                       | $O(b^m)$              | $O(bm)$               |
| Breadth-first       | FIFO queue               | yes                  | if $c$ uniform           | $O(b^d)$              | $O(b^d)$              |
| Iterative deepening | DFS, depth $1, 2, \dots$ | yes                  | if $c$ uniform           | $O(b^d)$              | $O(bd)$               |
| Uniform-cost        | priority $g(n)$          | if $c \geq \epsilon$ | yes                      | $O(b^{C^*/\epsilon})$ | $O(b^{C^*/\epsilon})$ |
| Greedy              | priority $h(n)$          | if no cycles         | no                       | $O(b^m)$              | $O(b^m)$              |
| A*                  | priority $f(n)$          | yes                  | if $h$ admissible (tree) | $O(b^m)$              | $O(b^m)$              |

# Creating admissible heuristics

Most of the work in solving hard search problems optimally is in finding admissible heuristics.

Admissible heuristics can be derived from the exact solutions to **relaxed problems**, where new actions are available.



## Dominance

- If  $h_1$  and  $h_2$  are both admissible and if  $h_2(n) \geq h_1(n)$  for all  $n$ , then  $h_2$  **dominates**  $h_1$  and is **better** for search.
- Given any admissible heuristics  $h_a$  and  $h_b$ ,

$$h(n) = \max(h_a(n), h_b(n))$$

is also admissible and dominates  $h_a$  and  $h_b$ .

## Example: the 8-puzzle

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

Start State

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

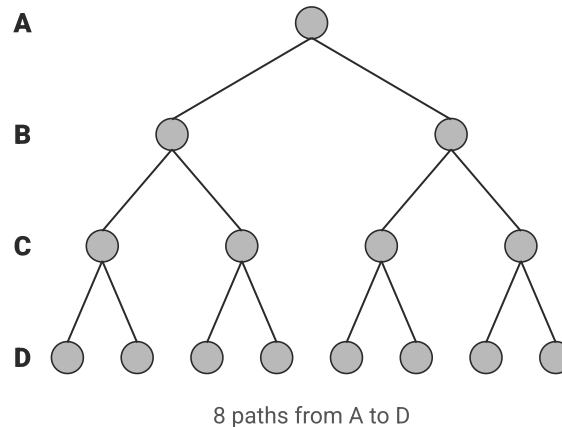
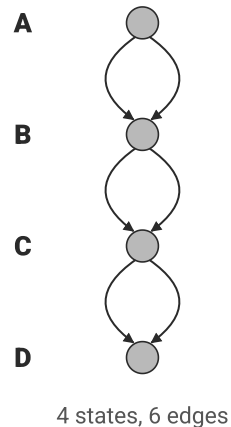
Goal State

- $h_1(n)$ , the number of misplaced tiles:  $h_1 = 8$  above.
- $h_2(n)$ , the sum of the Manhattan distances of the tiles to their goal positions:  
 $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$ .

Both are admissible, and  $h_2$  **dominates**  $h_1$ : a misplaced tile is at least one move away from its goal position, so  $h_1(n) \leq h_2(n)$  everywhere. A\* expands fewer nodes with  $h_2$ .

# Graph search

# Graph search



A state can be reached by several paths, and the tree holds one node per path: two edges at each level make  $2^3 = 8$  paths from A to D, for 4 states.

The failure to detect **repeated states** can turn a linear problem into an exponential one. It can also lead to non-terminating searches.

Redundant paths and cycles can be avoided by **keeping track** of the states that have been **explored**. This amounts to grow a tree directly on the state-space graph.

**function** GRAPH-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

*closed*  $\leftarrow$  an empty set

*fringe*  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

**loop do**

**if** *fringe* is empty **then return** failure

*node*  $\leftarrow$  REMOVE-FRONT(*fringe*)

**if** GOAL-TEST(*problem*, STATE[*node*]) **then return** *node*

**if** STATE[*node*] is not in *closed* **then**

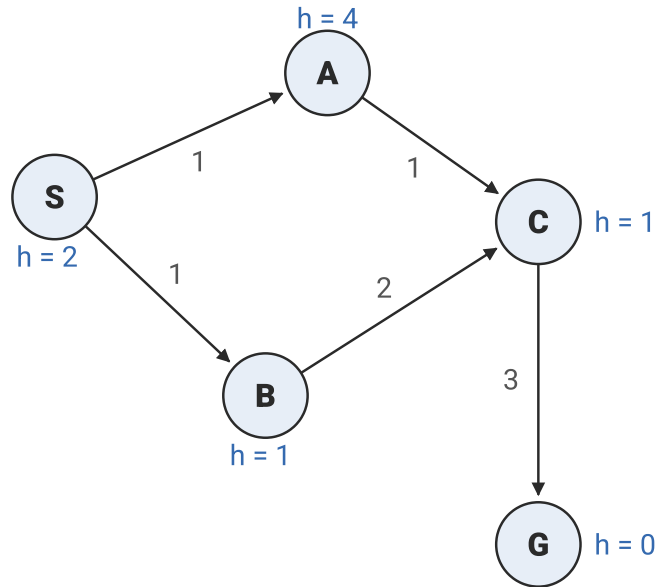
        add STATE[*node*] to *closed*

*fringe*  $\leftarrow$  INSERTALL(EXPAND(*node*, *problem*), *fringe*)

**end**

## A\* graph-search gone wrong?

- We start at *S* and *G* is a goal state.
- Which path does graph search find?



## Answer

$h$  is admissible, but it is **not consistent**:  $h(A) = 4 > c(A, a, C) + h(C) = 2$ .

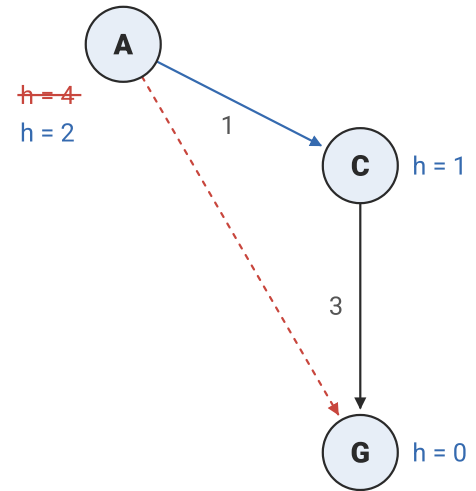
Graph search expands  $S$ , then  $B$  ( $f = 1 + 1$ ), then  $C$  through  $B$  ( $f = 3 + 1$ ), which **closes**  $C$  with  $g(C) = 3$ . When  $A$  is expanded ( $f = 1 + 4$ ), the cheaper path to  $C$  ( $g(C) = 2$ ) is discarded, since  $C$  is already closed.

A\* returns  $S, B, C, G$ , of cost 6, instead of  $S, A, C, G$ , of cost 5.

## Consistent heuristics

A heuristic  $h$  is consistent if for every  $n$  and every successor  $n'$  generated by any action  $a$ ,

$$h(n) \leq c(n, a, n') + h(n').$$



Consequences of consistent heuristics:

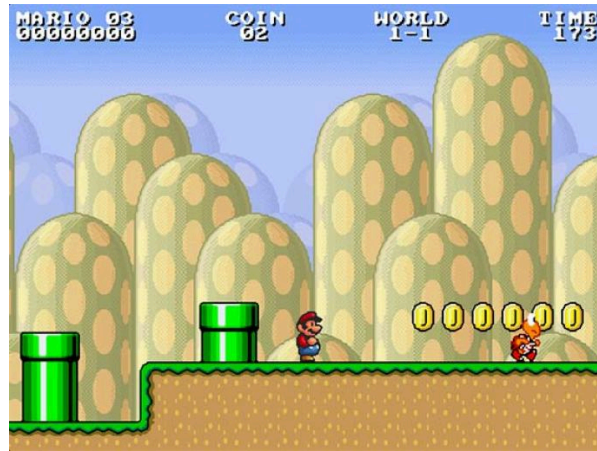
- $f(n)$  is non-decreasing along any path.
- $h(n)$  is admissible.
- With a consistent heuristic, graph-search A\* is optimal.

## When $h$ is not consistent

Graph search closes a state the first time it is expanded, possibly through a suboptimal path. Two ways out:

- use a **consistent** heuristic, so that a state is first expanded along its cheapest path;
- **re-expand** a closed state when it is reached with a lower  $f$ , which restores optimality for any admissible  $h$ , at the price of re-expansions.

# Recap example: Super Mario



- **Task environment?**
  - the performance measure  $V$ , the environment, the actuators  $\mathcal{A}$ , the sensors  $\mathcal{P}$ ?
- **Type** of environment?
  - observable, deterministic, episodic, static, discrete, single-agent, known?
- **Search problem?**
  - $s_1$ ,  $\text{actions}(s)$ ,  $\text{result}(s, a)$ ,  $G$ ,  $c(s, a, s')$ ?
- **Good heuristic**  $h(n)$ ?



A\* in action

# Summary

- A **search problem** is a state space  $\mathcal{S}$ , an initial state  $s_1$ , actions  $\text{actions}(s)$ , a transition model  $\text{result}(s, a)$ , goal states  $G$  and step costs  $c$ . A solution is a sequence  $a_{1:T}$  reaching  $G$ , optimal when its path cost is minimal. Choosing what to keep in a state is what makes the search feasible.
- A **strategy** is the order in which the fringe is expanded: LIFO for depth-first, FIFO for breadth-first,  $g(n)$  for uniform-cost,  $h(n)$  for greedy,  $f(n) = g(n) + h(n)$  for A\*.
- Only **uniform-cost** and **A\*** order by cost, and only they are optimal in general. Breadth-first and iterative deepening return the **shallowest** goal, which is the cheapest one only under uniform step costs.
- $h$  is **admissible** if  $h(n) \leq h^*(n)$ , which makes tree-search A\* optimal, and **consistent** if  $h(n) \leq c(n, a, n') + h(n')$ , which makes graph-search A\* optimal. A **dominating** heuristic expands fewer nodes.
- **Graph search** keeps a closed set, so a state is expanded once, along the first path that reaches it. The price is that optimality then needs consistency, or re-expanding closed states reached with a lower  $f$ .

